ANALYTICS ENGINEERING · PORTFOLIO PROJECT

OneOnOne Analytics dbt Model Walkthrough

A layer-by-layer, model-by-model explanation of the transformation pipeline — the business purpose, the grain, the joins, the logic, and the engineering decisions behind every model.

Stack: **Supabase (Postgres)** → Python ingestion → **Snowflake** → **dbt** (staging · intermediate · marts) → MetricFlow (next)
Scope of this document: 9 staging views · 3 intermediate views · 5 mart tables · 75 data tests I Prepared for interview discussion

0 · The big picture — why a layered model

This is an **ELT** pipeline: we **E**xtract from Supabase and **L**oad raw into Snowflake first, then **T**ransform inside the warehouse with dbt. Transformation is split into layers, each with exactly one job. The guiding principle is **separation of concerns**: a problem (typing, a reusable join, a business definition) is solved in exactly one place, so it can be tested once and reused everywhere downstream.

Layer	Job	Materialization	Why that materialization
RAW	Untouched landing of source rows (ingestion writes here)	Table (by Python)	Durable copy of source; everything is VARCHAR so a schema change never breaks the load
staging	Clean: cast types, rename to snake_case, 1 model per source table	**view**	Always fresh, zero storage cost; it is just a thin typed lens over RAW
intermediate	Reusable "glue": joins & derivations needed by more than one mart	**view**	Throwaway building blocks; not queried by consumers, so no need to persist
marts	The business-facing **data model**: dimensions & facts	**table**	Queried often by the agent/BI; persisting makes reads fast and stable
MetricFlow	The actual metrics/KPIs, defined *on top of* the marts	semantic layer	Metrics are a contract, not a table — they compose dimensions + measures on demand

Key distinction to state in an interview: the **marts are the data model** (`dim_*` = entities, `fct_*` = events). They are *not* the metrics. Metrics (`active_managers` , `mrr` , `churn_rate` …) live in the **semantic layer** on top of the marts. Conflating "mart" with "metric" is a common mistake.

CROSS-CUTTING MECHANICS WORTH NAMING

- **Schema-per-layer + RBAC.** Snowflake has one schema per layer (`RAW / STAGING / INTERMEDIATE / MARTS`), owned by `SYSADMIN` . dbt runs as a least-privilege role `ONEONONE_TRANSFORMER` that can *read* RAW and *build* in the modeled schemas — but cannot create schemas itself. A custom `generate_schema_name` macro routes each folder to its schema verbatim.
- **The DAG.** Models reference each other with `{{ ref('…') }}` ; dbt builds a dependency graph and runs them in the correct order automatically. No hardcoded table names.
- **Tests as data contracts.** Every primary key has `unique` + `not_null` ; foreign keys have `relationships` ; enum columns have `accepted_values` . A failing test stops the build.

1 · Staging layer 9 views · schema STAGING

Business purpose: make the messy source trustworthy. Supabase mixes naming conventions (older tables are camelCase like `"createdAt"`, newer ones snake_case), and RAW stores *everything as VARCHAR*. Staging is the one place we fix that, so every downstream model can assume clean, correctly-typed, consistently-named columns.

THE RULES EVERY STAGING MODEL FOLLOWS

- **One model per source table**, strictly 1:1 — **no joins, no business logic**. This keeps the layer predictable and means a bug is always traceable to a single source.
- **Rename** to `snake_case`; quote the camelCase source columns (`"boardId"`) so Snowflake preserves their case when reading.
- **Cast** types explicitly: timestamps to `TIMESTAMP_TZ` (preserves the UTC offset from Supabase's `timestamptz` — chosen over `NTZ/LTZ` which lose or remap it), dates to `DATE`, booleans to `BOOLEAN`.
- **Audit column** `_dbt_loaded_at = current_timestamp()` on every row for lineage.
- **Source-aware**: read via `{{ source('supabase','table') }}` (named `supabase` so a future `dodo` payments source slots in as a second named source).

REPRESENTATIVE MODEL — `STG_ENTRIES`

```
with source as (
    select * from {{ source('supabase', 'entries') }}
),
renamed as (
    select
        "id"::varchar             as entry_id,
        "boardId"::varchar        as board_id,
        "employeeId"::varchar     as employee_id,
        "entryDate"::date         as entry_date,   -- business date (when work happened)
        "status"::varchar         as status,
        "createdAt"::timestamp_tz as created_at,   -- audit date (when logged)
        current_timestamp()       as _dbt_loaded_at
    from source
)
select * from renamed
```

Why `entry_date` vs `created_at` matters: a business rule says all business metrics use `entryDate` (when the work happened), and `createdAt` is for audit/pipeline only. Keeping both, clearly named, lets downstream models pick the right one and never mix them.

Model	Source casing	Notable casts / tests
stg_users	camelCase	role → `accepted_values` [manager, reportee]
stg_boards	camelCase	timestamps → TIMESTAMP_TZ; PK board_id
stg_memberships	camelCase	full-refresh table; PK membership_id
stg_entries	camelCase	entry_date → DATE; status enum
stg_announcements	camelCase	PK announcement_id
stg_announcement_reactions	camelCase	reaction → `accepted_values`
stg_subscriptions	snake_case	is_complimentary → BOOLEAN; plan & status enums
stg_support_requests	camelCase	type → request_type
stg_usage_events	snake_case	append-only; PK usage_event_id

Testing: `unique` + `not_null` on all 9 primary keys, plus `accepted_values` on every enum (role, visibility, plan, status, reaction). These catch duplicate keys, orphaned rows, and unexpected enum values at the earliest possible point.

2 · Intermediate layer 3 views · schema INTERMEDIATE

Business purpose: centralize the join/derivation logic that *more than one mart needs*, so a business rule (self-entry exclusion, the MRR definition, team size) is written and tested **once**. Without this layer, the same join would be copy-pasted into three marts and drift apart when the rule changes. These are `view`s — pure building blocks, never queried directly by a consumer.

int_entries_enriched

grain: one row per entry

Business question it serves: "for any entry, whose work is it, is it the manager logging their own work, and is it that employee's very first published entry?"

Inputs / ERD edge: `stg_entries` ⋈ `stg_boards` on `board_id` (the entries table has no `manager_id`, so we pull it from the board).

Key logic:

- **`is_self_entry`** — the business rule "manager self-entries are excluded from team metrics". Computed as `coalesce(employee_id = manager_id, false)`; the `coalesce` guards the case of a missing board (left join → null) so the flag is never null.
- **`is_first_entry`** — true only for an employee's earliest *published* entry. We `row_number()` over *only published* rows, then map rank 1 → true.

```
-- only published entries are eligible to be "first"
published_ranked as (
    select entry_id,
           row_number() over (partition by employee_id
                              order by entry_date, created_at) as published_rank
    from joined
    where status = 'published'
)
-- ...later, null-safe boolean for every row:
, coalesce(published_ranked.published_rank = 1, false) as is_first_entry
```

> **Why rank in a separate CTE filtered to `published`:** if we ranked all statuses, a draft could be numbered "first". Filtering first guarantees a draft can never win; the `left join` back + `coalesce` makes non-published rows `false` rather than null.

int_board_team_size

grain: one row per board

Business question: "how many real team members (reportees) are on each board?"

Inputs / ERD edge: `stg_memberships` ⋈ `stg_boards` on `board_id`.

Key logic: count members **excluding the manager** (`where user_id != manager_id` — the manager isn't their own team member). A `left join` from boards plus `count(distinct user_id)` makes empty boards return **0**, not null.

> **Why `count(distinct)`:** it is defensive against a member appearing twice; counting a nullable joined column also yields 0 for boards with no members (count ignores nulls).

int_subscriptions_current

grain: one row per user

Business question: "what is each user's *current* plan, are they paying, and how much MRR do they contribute?"

Inputs / ERD edge: `stg_subscriptions` (single table; `user_id → users`).

Key logic — pick one row per user: a user can have many subscription rows over time, so we deduplicate with `qualify row_number()`: prefer an `active` row, then the latest `current_period_end`, then the most recently updated.

```
qualify row_number() over (
    partition by user_id
    order by iff(status = 'active', 1, 0) desc,   -- active wins
             current_period_end desc nulls last,  -- else latest period
             updated_at        desc nulls last    -- final tiebreak
) = 1
```

Derived measures (the MRR business rule): `is_paying` = active **AND** not complimentary **AND** on a paid plan; `mrr_amount` applies the same gate then maps plan → price.

Open business input: plan prices (`pro=$29` , `pro_plus=$99`) are **placeholders**. Because the rule "free = $0 MRR" must hold and prices will be confirmed, the clean next step is a **pricing macro/seed** so the number lives in exactly one place (it currently appears here and in `fct_subscriptions`).

3 · Marts layer 5 tables · schema MARTS

Business purpose: the curated, business-facing **data model** the AI analyst and BI tools query. `dim_*` are **dimensions** (entities you slice by — managers, teams). `fct_*` are **facts** (events you measure — entries, subscription changes). One mart (`mart_manager_health`) is a pre-aggregated snapshot. All are `table`s for fast, stable reads.

The fan-out problem (the headline technical point here): a manager has many boards, many members, many entries, many subscription rows. Joining them all at once would multiply rows and corrupt every count. The fix used in every dimension: **pre-aggregate each related table to the dimension's grain in its own CTE, then `left join` those 1:1**. This is exactly the work the intermediate layer centralizes.

dim_managers

grain: one row per manager (users where role = 'manager')

Answers: who are our managers, what plan are they on, are they paying, how big is their team, have they activated, and how active are they?

Reuses: `int_board_team_size` (team size), `int_entries_enriched` (entry stats), `int_subscriptions_current` (plan / is_paying).

Key logic:

- **Activation** (the north-star signal): `is_activated` = at least one *team* member has at least one `published` entry — computed over `int_entries_enriched` with `is_self_entry = false`. "Board created ≠ activated."
- **entry_count / last_entry_at** aggregate only non-self entries (the self-entry rule again, applied for free because the flag lives upstream).
- **is_paying** is taken from the intermediate model — caught a bug here in review: a *free* plan can be "active & non-complimentary", so paying must also require a **paid** plan.

```
-- each related table pre-aggregated to manager grain, then joined 1:1 (no fan-out)
left join current_subscription on managers.user_id = current_subscription.user_id
left join team_size            on managers.user_id = team_size.manager_id
left join entry_stats          on managers.user_id = entry_stats.manager_id
```

dim_teams

grain: one row per board (team)

Answers: per team — how big is it, when was it created, and what plan is the owning manager on?

Reuses: `int_board_team_size` (per-board team size directly — same grain, a clean fit), `int_subscriptions_current` (manager's plan via `manager_id = user_id`).

Test of note: `relationships` on `manager_id → dim_managers` — every team's manager must exist as a manager, a referential-integrity guarantee.

fct_entries

grain: one row per entry

Answers: the analyzable grain of "work logged" — category, status, and the `is_self_entry` / `is_first_entry` flags ready for aggregation.

Design: a **thin** `select` from `int_entries_enriched`. This is the clearest demonstration of *why* the intermediate layer exists: the expensive join + window were done once upstream; the fact is just the business-facing projection of it.

Tests: status → `accepted_values` [draft, pending_approval, published] (the entry lifecycle); `relationships` on `board_id → dim_teams`.

fct_subscriptions

grain: one row per subscription record (full history, not just current)

Answers: the revenue event stream — per row MRR contribution and what changed vs the prior row.

Key logic — `transition_type`: rank plans (`free<pro<pro_plus`), use `lag()` over the user's history to see the previous plan/status, then classify `new / upgrade / downgrade / churn / unchanged`.

```
lag(plan_rank) over (partition by user_id order by created_at, updated_at) as prev_plan_rank
-- ...
case when prev_plan_rank is null then 'new'
     when status in ('canceled','expired') and prev_status not in ('canceled','expired') then 'churn'
     when plan_rank > prev_plan_rank then 'upgrade'
     when plan_rank < prev_plan_rank then 'downgrade'
     else 'unchanged' end as transition_type
```

Why this fact keeps all rows (unlike `int_subscriptions_current` which keeps one): MRR trend, upgrade/downgrade/churn analysis all need the *history*. The intermediate model answers "current state"; the fact preserves "the whole timeline".

mart_manager_health

grain: one row per manager — snapshot (recomputed each run, no history kept)

Answers: "who is at churn risk *right now*?" — a ready-to-read early-warning for the agent.

Reuses: `dim_managers` (last_entry_at, team_size, plan) + `int_entries_enriched` (entries in the trailing 30 days).

Key logic: exposes both the **raw components** (`entries_30d` , `days_since_last_entry` , `team_size` , `current_plan`) **and a derived verdict** `health_status` , so the agent can cite a number or a label:

```
case when last_entry_at is null
       or datediff(day, last_entry_at, current_date()) > 30 then 'churning'
     when datediff(day, last_entry_at, current_date()) > 14 then 'at_risk'
     else 'healthy' end as health_status
```

"Snapshot" decision: one row per manager, overwritten each run — it answers "status today". Keeping daily *history* (one row per manager per day) would need a date spine and is deferred until a trend is actually required. A deliberate scope choice, not an oversight.

4 · Cross-cutting concerns

Business rules encoded in the model (state these confidently)

Rule	Definition	Where it lives
Manager self-entries	An entry where `employee_id = board.manager_id` is the manager logging their own work — excluded from all team-level metrics.	`int_entries_enriched.is_self_entry`, applied in dim_managers & mart_manager_health
Activation	A manager is "activated" when ≥1 team member has ≥1 `published` entry. Board created ≠ activated.	`dim_managers.is_activated`
MRR	Only `status='active'` AND `is_complimentary=false` AND a paid plan contribute. Free = $0.	`int_subscriptions_current`, `fct_subscriptions`
Entry lifecycle	`draft → pending_approval → published`; published is terminal.	enforced via `accepted_values` on status
Business vs audit date	`entry_date` for business metrics; `created_at` for pipeline/audit. Never mixed.	staging keeps both, clearly named

Testing strategy — tests as a data contract

75 data tests across the project. Each category guards a specific failure mode:

- unique + not_null on every primary key → guarantees the stated **grain** (no duplicates, no missing keys).
- relationships on foreign keys (`manager_id→dim_managers`, `board_id→dim_teams`) → catches **orphaned rows** / referential breaks.
- accepted_values on enums → catches an **unexpected category** the app starts emitting before it silently corrupts a metric.
- not_null on derived booleans/measures → proves the `coalesce` null-handling actually works.

Run with `dbt build`, which interleaves model runs and their tests in DAG order and fails the whole build if any contract is violated — so bad data cannot reach the marts.

Engineering decisions, summarized

Decision	Rationale
ELT (load raw, transform in-warehouse)	Snowflake compute is cheap & scalable; RAW is a replayable source of truth
staging/intermediate = views, marts = tables	Views stay fresh with no storage; tables make frequent consumer reads fast
Dedicated INTERMEDIATE schema + least-privilege role	Clean RBAC: schemas owned by SYSADMIN, dbt builds with only the rights it needs
Pre-aggregate-then-join in dimensions	Eliminates fan-out; keeps every count correct at the stated grain
`qualify row_number()` for "current" picks	Deterministic dedup to one row per entity with explicit tiebreakers
Idempotent ingestion (load-then-swap)	Atomic full refresh; a crash leaves the previous complete data in place

5 · Interview Q&A — likely questions

Q. Why have an intermediate layer at all — why not join inside each mart?
Because the same join/rule (self-entry, team size, current subscription) is needed by 2–3 marts. Centralizing it means it's written and tested once; change the rule in one place and every mart updates. With one mart it's optional — it earns its place the moment logic repeats.

Q. What's the difference between a mart and a metric?
Marts are the **data model** — dimensions and facts you can slice and aggregate. Metrics/KPIs are a **semantic layer on top** (MetricFlow): named, reusable calculations like `mrr` or `active_managers`. A dimension like `dim_managers` isn't a metric; it's what metrics are computed *over*.

Q. How do you avoid fan-out when a manager has many boards/entries/subs?
Pre-aggregate each related table to the dimension's grain in its own CTE, then `left join` those 1:1 to the base entity. Never join multiple one-to-many tables directly — that multiplies rows and breaks counts.

Q. How do you pick a single "current" subscription per user?
`qualify row_number() over (partition by user_id order by active-first, latest period_end, latest updated)= 1`. Explicit, deterministic, with tiebreakers — and a fallback to the most recent row when none is active.

Q. How is grain guaranteed and how would you catch a regression?
Grain is declared in a comment and **enforced** by `unique` + `not_null` on the key. If a join ever fans out, the `unique` test fails and `dbt build` stops — the contract catches it.

Q. Why `TIMESTAMP_TZ` and why cast in staging?
Supabase sends `timestamptz`; `TIMESTAMP_TZ` preserves the UTC offset (NTZ/LTZ would drop or remap it). Casting in staging means every downstream model gets correct types for free and the rule lives in one place.

Q. What would you improve next?
Move plan prices into a pricing macro/seed (single source of truth for MRR); add the MetricFlow semantic layer; and add fail-loud + row-count reconciliation to ingestion (tracked in the improvements backlog).

Generated for the OneOnOne Analytics project · reflects 9 staging views, 3 intermediate views, 5 mart tables, 75 passing tests (`dbt build` green).